

Python Data structure

Part 1: Built-in structures

Structure	Applications shown
List	To-do manager, matrix addition
Tuple	Returning multiple values, GPS coordinates as dict keys
Set	Removing duplicates, mutual friends
Dictionary	Word frequency, student result database
<code>deque</code> , <code>Counter</code> , <code>defaultdict</code>	Rotating a deque, most common words, grouping words by first letter

Many More

Part 2: Structures built from scratch

Structure	Applications shown
Stack	Balanced brackets, postfix evaluation, string reversal, undo
Queue (linked nodes)	Printer spooler, round-robin CPU scheduling
Singly Linked List	Music playlist
Doubly Linked List	Browser back/forward history
Binary Search Tree	Student database indexed by roll number
Min-Heap	Task scheduler, heap sort, emergency-room priority
...	...

Python Data Structures and Their Role in Computer Applications

A **data structure** is a way of organizing and storing data so that it can be **accessed, processed, modified, and managed efficiently**.

In Python, the most commonly used built-in data structures are:

Data Structure	Example	Main Property	Typical Application
List	<code>[10, 20, 30]</code>	Ordered, mutable	Student marks, shopping cart
Tuple	<code>(10, 20, 30)</code>	Ordered, immutable	Fixed records, coordinates
Set	<code>{10, 20, 30}</code>	Unique elements	Removing duplicates
Dictionary	<code>{"id":101, "name":"Amit"}</code>	Key-value pairs	Database-like records
Array	<code>array('i',[10,20,30])</code>	Homogeneous data	Numerical processing
String	<code>"Python"</code>	Sequence of characters	Text processing



M5 Applications of data structure

- **Student Management System.**

1. Python List — Student Marks

A **List** stores multiple values and allows duplicate values.

</> Python

```
# List of marks of a student
marks = [85, 72, 90, 68, 85]

print("Marks:", marks)
print("First mark:", marks[0])
print("Last mark:", marks[-1])

# Add a new mark
marks.append(95)

print("Updated marks:", marks)
print("Number of subjects:", len(marks))
```

Output:

```
Marks: [85, 72, 90, 68, 85]
First mark: 85
Last mark: 85
Updated marks: [85, 72, 90, 68, 85, 95]
Number of subjects: 6
```

2. Python Dictionary — Student Information

A Dictionary stores data in key-value pairs.

`</>` Python

```
# Student information
student = {
    "name": "Rahul",
    "roll_no": 101,
    "course": "B.Tech CSE",
    "semester": 5,
    "cgpa": 8.4
}

print("Student Name:", student["name"])
print("Roll Number:", student["roll_no"])
print("Course:", student["course"])
print("CGPA:", student["cgpa"])

# Update CGPA
student["cgpa"] = 8.7

# Add new information
student["city"] = "Bhopal"

print("\nUpdated Student Information:")
print(student)
```

Output:

Student Name: Rahul

Roll Number: 101

Course: B.Tech CSE

CGPA: 8.4

Updated Student Information:

```
{'name': 'Rahul', 'roll_no': 101, 'course': 'B.Tech CSE',
 'semester': 5, 'cgpa': 8.7, 'city': 'Bhopal'}
```

3. Python Set — Unique Subjects

A **Set** stores **unique values** and automatically removes duplicates.

`</>` Python

```
# Subjects registered by students
subjects = {
    "Python",
    "Machine Learning",
    "Cloud Computing",
    "Python",
    "Data Science"
}

print("Subjects:", subjects)

# Add a new subject
subjects.add("Artificial Intelligence")

print("Updated Subjects:", subjects)

# Check whether a subject exists
if "Python" in subjects:
    print("Python is available")
```

Quick Comparison

Data Structure	Example	Duplicate Values	Main Use
List	<code>[85, 72, 90]</code>	✓ Allowed	Collection of items
Dictionary	<code>{"name": "Rahul"}</code>	Keys ✗	Key-value information
Set	<code>{"Python", "AI"}</code>	✗ Not allowed	Unique values
Array	<code>array('i', [85, 72, 90])</code>	✓ Allowed	Numerical data

4. Python Array — Student Marks

For numerical data, we can use a Python array.

</> Python

```
from array import array

# Integer array containing marks
marks = array('i', [85, 72, 90, 68, 95])

print("Marks:", marks)

# Access an element
print("First mark:", marks[0])

# Add a mark
marks.append(88)

print("Updated marks:", marks)

# Calculate total
total = sum(marks)

# Calculate average
average = total / len(marks)

print("Total Marks:", total)
print("Average Marks:", average)
```

- from array import array
- **# LIST:** marks of students
- marks = [85, 72, 90, 68, 85]
- **# DICTIONARY:** student information
- student = {
 - "name": "Rahul",
 - "roll_no": 101,
 - "course": "B.Tech CSE"
- }
- **# SET:** unique subjects
- subjects = {
 - "Python",
 - "AI",
 - "Python",
 - "Cloud Computing"
- }
- **# ARRAY:** numerical marks
- marks_array = array('i', [85, 72, 90, 68, 85])